Performance Testing

Date	23 June 2026
Team ID	SWTID-2026-7037
Project Name	OptiCrop: Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Apache JMeter
Type of Testing	Load Testing
Target Module / API	/predict endpoint (FindYourCrop crop prediction route)
Test Environment	Local (Flask development server)
Test Date	20 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome

1	Single user submits prediction form once	1	10	Prediction returned instantly with no errors
2	Multiple users submit predictions concurrently (normal load)	20	60	All requests processed with response time < 2s
3	Higher concurrent load (peak usage simulation)	50	120	System remains stable, response time < 5s
4	Sustained load over longer duration	30	300	No memory leaks or crashes; consistent response times

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (Pass / Fail)	Remarks
1	Response Time (Avg)	< 2 seconds	0.8 seconds	Pass	Model inference is lightweight (Logistic Regression), keeping response time low
2	Response Time (Max)	< 5 seconds	1.9 seconds	Pass	Max observed under peak load (50 virtual users)
3	Throughput (Req/sec)	—	24 req/sec	Pass	Measured during 50-user concurrent test
4	Error Rate	< 1%	0%	Pass	No failed requests observed across all test scenarios
5	CPU Utilization	< 80%	45%	Pass	Local machine, single Flask dev server process
6	Memory Utilization	< 80%	38%	Pass	Model and dataset are small, minimal memory footprint

Step 4: Observations & Analysis Key

Findings:

The OptiCrop application performed well under both normal and peak simulated load. Since the trained Logistic Regression model is lightweight and inference is computationally simple, response times remained well within target thresholds even at higher concurrency.

Bottlenecks Identified:

The Flask development server is single-threaded by default, which could become a bottleneck under much higher concurrent traffic than tested here (e.g., hundreds of simultaneous users) in a real production environment**.**

Optimization Steps Taken:

For future production deployment, the team recommends running the app with a production-grade WSGI server (e.g., Gunicorn) behind a reverse proxy (e.g., Nginx), and enabling multiple worker processes to handle higher concurrent load**.**

Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):